Toward an Efficient General-Purpose Time-Series Model

Progress report: TimeMixer, TimeMixer++, SparseTSF and SparseTSFPlus on the TimeMixer++ task suite

23 October 2026 · prepared for the biweekly supervision meeting

> Every number in the tables of this report is generated from the per-run logs in the public repository (`BenguBarisBalkan/efficient-general-purpose-timeseries-models`) by `TimeMixer/benchmarks/tools/build_progress_document.py`; none is typed by hand. **Local** numbers were measured on the machine described in Section 1.3 under one shared protocol. **Published** numbers are copied from the TimeMixer and TimeMixer++ papers, were produced on other hardware with the authors' own tuning, and are shown in *italics* in separate columns. The two are never mixed in one cell.

Abstract

This thesis asks how efficient general-purpose time-series models are, and whether a model can be built that is both efficient and capable across the eight task families of the TimeMixer++ benchmark: long-term and short-term forecasting, imputation, few-shot and zero-shot forecasting, anomaly detection and classification. We compare three published models: TimeMixer (ICLR 2024), TimeMixer++ (ICLR 2025; reimplemented from the paper, since no official code exists) and SparseTSF (ICML 2024, about 1,000 parameters).

Over the last two weeks the comparison went from SparseTSF-only on most tasks to every task run locally for TimeMixer and SparseTSF. We then tested twelve modules on top of SparseTSF, each with a prediction recorded before its runs. Five were adopted (counting the refinement gate), five rejected, and two remain candidates.

The adopted modules form **SparseTSFPlus**, one fixed configuration with 37 to 8,661 parameters outside classification. Over three seeds it beats SparseTSF beyond run-to-run noise on five tasks:

Task	Gain over SparseTSF (3 seeds)
PEMS	−24%
Imputation	−16%
M4	−6%
Few-shot	−1.4%
Long-term	−1.0%

On every forecasting task it is within 0.7% of choosing modules per task. It uses 1.35× the energy of SparseTSF and about one fourteenth of TimeMixer's. Against TimeMixer it is ahead on few-shot and zero-shot transfer, and behind on long-term, M4, PEMS, imputation and classification. Repeating runs over seeds also showed that several single-seed conclusions in imputation and anomaly detection were noise, and those are corrected here.

1. Introduction

1.1 Research question

General-purpose time-series models, sometimes called time-series pattern machines, use one architecture for many task families. They are trained per task and dataset; they are not pretrained foundation models such as TimesFM or Chronos, although the zero-shot task probes a similar property. The state of the art in this family, TimeMixer++, reaches strong accuracy with a multi-scale, attention-based backbone; in this study TimeMixer and TimeMixer++ use 10^4 to 10^7 parameters. The question of this thesis is how much of that capacity is needed: **can a**

model of a few hundred to a few thousand parameters be made capable across all eight tasks, with one configuration?

1.2 Models

- **TimeMixer** (Wang et al., ICLR 2024). Decomposes each series into seasonal and trend parts at several temporal scales and mixes them across scales with MLPs. Upstream code; run locally with its published scripts where they exist (long-term, M4, PEMS), otherwise with its own long-term architecture.
- **TimeMixer++** (Wang et al., ICLR 2025). Adds multi-resolution "time imaging" by FFT-selected periods, dual-axis attention, and multi-scale mixing. No official code is public, so this study reimplemented it from the paper. The reimplementation is numerically unstable at the paper's learning rate and was only stable at `lr = 5e-5`, which undertrains it.
- **SparseTSF** (Lin et al., ICML 2024). Folds each channel's input into a grid of *phase × period count* and applies one small linear map across periods, so a whole forecaster costs tens to thousands of parameters. It is the baseline the modules of Section 3 are added to.

1.3 Protocol and setup

- **Tasks, datasets and metrics** follow the TimeMixer++ paper. Deviations are listed in Section 2.3.
- **Hardware:** one Windows laptop with a 4 GB NVIDIA GPU (RTX 3050 Ti) and 16 GB of RAM. Batch sizes are reduced from the papers' values where memory requires.
- **Hyperparameters:** each local number uses the model's own published settings where they exist. A run with a module copies the settings of the SparseTSF run it is compared with, so the two differ only in the module.
- **Selection** uses validation loss, never test loss. Imputation, anomaly detection and classification early-stop on the test split in the upstream code shared by all models; their results are therefore a sensitivity analysis.
- **Seeds:** every run uses seed 2021 unless stated otherwise. The headline comparisons were repeated with seeds 2022 and 2023 (Section 4.3). A difference counts as a *finding* only if it has the same sign on all three seeds and its mean exceeds twice the standard deviation of the per-seed differences.

1.4 Adapting SparseTSF to every task

SparseTSF was published for long-term forecasting only. Before any comparison or module could be tested, it was given heads for the other task families, built from its own components:

- **Reconstruction** (imputation, anomaly detection): fold the input and map periods back to periods with a square linear map, instead of input periods to future periods.
- **Classification:** aggregate with SparseTSF's convolution, flatten, and apply one linear layer.
- **Forecasting:** delegates to the untouched original code, so published SparseTSF numbers are reproduced exactly.

The classification head is the only place SparseTSF is not small, because its size scales with channels × input length × classes (up to 970,712 parameters).

2. Three-model comparison

The comparison tables follow the layout of the TimeMixer++ paper's main results:

- **Long-term, imputation and few-shot:** the four ETT datasets are averaged into "ETT (Avg)".
- **M4 and PEMS:** averaged over their subsets or datasets.
- **Anomaly detection and classification:** averages only.

Model columns are **local** runs; columns marked "(paper)" are **published** values in *italics*. Among local models, the best value per cell is **bold** and the second best underlined; published values are not ranked. "partial" means not every cell of that row finished; "n/c" means not comparable (the published imputation results use a 1,024-step input, Section 2.3). Published long-term values are averaged from the papers' full-results tables. For TimeMixer++ ETT this gives an MAE of 0.377, where the paper's own summary table lists 0.399. Every number per dataset is in Appendix A.

2.1 State at the start of the reporting period (11 October 2026)

Table 1 shows every result that existed on 11 October 2026. Each log is read from git at commit `72040e6`, so a cell counts only if its run had finished by then. At that point only SparseTSF had been run on the per-task benchmarks. TimeMixer had been run locally only on long-term forecasting and on imputation, and TimeMixer++ only on long-term forecasting. Appendix B gives the same state per dataset.

Table 1. Three-model comparison on 11 October 2026 (commit `72040e6`).

Benchmark	TimeMixer		TimeMixer++		SparseTSF		*TimeMixer (paper)*		*TimeMixer++ (paper)*	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Long-term forecasting (MSE / MAE, mean of 4 horizons, input 96)										
ETT (Avg)	**0.374**	0.393	0.492	0.462	0.374	**0.389**	*0.367*	*0.389*	*0.349*	*0.377*
Weather	**0.245**	**0.276**	0.276	0.304	0.274	0.294	*0.240*	*0.272*	*0.226*	*0.262*
Solar-Energy	0.232	0.298	partial		—		*0.216*	*0.280*	*0.203*	*0.258*
Electricity	0.183	0.272	—		—		*0.182*	*0.273*	*0.165*	*0.253*
Traffic	partial		—		—		*0.485*	*0.297*	*0.416*	*0.264*
Univariate short-term forecasting, M4 (average of 6 subsets)										
SMAPE	—		—		13.436		*11.723*		*11.448*	
MASE	—		—		1.825		*1.559*		*1.487*	
OWA	—		—		0.972		*0.840*		*0.821*	
Multivariate short-term forecasting, PEMS (average of 4 datasets, 96 → 12)										
MAE	—		—		34.13		*17.41*		*15.91*	
MAPE (%)	—		—		20.81		*10.59*		*10.08*	
RMSE	—		—		52.10		*28.01*		*27.06*	
Imputation (MSE / MAE, mean of 4 mask rates, input 96)										
ETT (Avg)	**0.065**	**0.155**	—		0.084	0.175	n/c		n/c	
Electricity	—		—		0.130	0.259	n/c		n/c	
Weather	**0.032**	**0.050**	—		0.034	0.058	n/c		n/c	
Few-shot forecasting, 10% of training data (MSE / MAE, mean of 4 horizons)										
ETT (Avg)	—		—		0.441	0.427	*0.453*	*0.445*	*0.396*	*0.421*
Weather	—		—		0.271	0.298	*0.242*	*0.281*	*0.241*	*0.271*
Electricity	—		—		partial		*0.187*	*0.277*	*0.168*	*0.271*
Zero-shot transfer, train → test (MSE / MAE, mean of 4 horizons)										
ETTh1 → ETTh2	—		—		0.360	0.390	*0.427*	*0.424*	*0.367*	*0.391*
ETTh1 → ETTm2	—		—		0.321	0.359	*0.361*	*0.397*	*0.301*	*0.357*
ETTh2 → ETTh1	—		—		0.453	0.449	*0.679*	*0.577*	*0.511*	*0.498*
ETTm1 → ETTh2	—		—		0.396	0.413	*0.452*	*0.441*	*0.417*	*0.422*
ETTm1 → ETTm2	—		—		0.296	0.328	*0.329*	*0.357*	*0.291*	*0.331*
ETTm2 → ETTm1	—		—		0.533	0.471	*0.554*	*0.478*	*0.427*	*0.448*
Anomaly detection and classification (averages)										
Anomaly F1 (%), 4 datasets	—		—		partial		—		*85.75*	

Benchmark	TimeMixer		TimeMixer++		SparseTSF		*TimeMixer (paper)*		*TimeMixer++ (paper)*	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Classification accuracy (%), 10 UEA datasets	—		—		60.90		—		*75.90*	

2.2 Current state (23 October 2026)

Over the reporting period TimeMixer was run locally on every task, SparseTSF on the three remaining long-term datasets and on MSL and SMAP, and the remaining imputation and few-shot cells were completed. Table 2 is the comparison as it stands; Table 3 counts the finished runs then and now.

Table 2. Three-model comparison on 23 October 2026.

Benchmark	TimeMixer		TimeMixer++		SparseTSF		*TimeMixer (paper)*		*TimeMixer++ (paper)*	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Long-term forecasting (MSE / MAE, mean of 4 horizons, input 96)										
ETT (Avg)	**0.374**	0.393	0.492	0.462	0.374	**0.389**	*0.367*	*0.389*	*0.349*	*0.377*
Weather	**0.245**	**0.276**	0.276	0.304	0.274	0.294	*0.240*	*0.272*	*0.226*	*0.262*
Solar-Energy	**0.232**	**0.298**	partial		0.386	0.371	*0.216*	*0.280*	*0.203*	*0.258*
Electricity	**0.183**	**0.272**	—		0.222	0.296	*0.182*	*0.273*	*0.165*	*0.253*
Traffic	partial		—		0.635	0.379	*0.485*	*0.297*	*0.416*	*0.264*
Univariate short-term forecasting, M4 (average of 6 subsets)										
SMAPE	**11.718**		—		13.436		*11.723*		*11.448*	
MASE	**1.563**		—		1.825		*1.559*		*1.487*	
OWA	**0.841**		—		0.972		*0.840*		*0.821*	
Multivariate short-term forecasting, PEMS (average of 4 datasets, 96 → 12)										
MAE	**17.51**		—		34.13		*17.41*		*15.91*	
MAPE (%)	**11.41**		—		20.81		*10.59*		*10.08*	
RMSE	**28.14**		—		52.10		*28.01*		*27.06*	
Imputation (MSE / MAE, mean of 4 mask rates, input 96)										
ETT (Avg)	**0.065**	**0.155**	—		0.084	0.175	n/c		n/c	
Electricity	**0.080**	**0.192**	—		0.130	0.259	n/c		n/c	
Weather	**0.032**	**0.050**	—		0.034	0.058	n/c		n/c	
Few-shot forecasting, 10% of training data (MSE / MAE, mean of 4 horizons)										
ETT (Avg)	0.513	0.462	—		**0.441**	**0.427**	*0.453*	*0.445*	*0.396*	*0.421*
Weather	**0.254**	**0.278**	—		0.271	0.298	*0.242*	*0.281*	*0.241*	*0.271*
Electricity	**0.209**	**0.292**	—		0.224	0.297	*0.187*	*0.277*	*0.168*	*0.271*
Zero-shot transfer, train → test (MSE / MAE, mean of 4 horizons)										
ETTh1 → ETTh2	0.367	0.393	—		**0.360**	**0.390**	*0.427*	*0.424*	*0.367*	*0.391*
ETTh1 → ETTm2	0.327	0.367	—		**0.321**	**0.359**	*0.361*	*0.397*	*0.301*	*0.357*
ETTh2 → ETTh1	0.639	0.548	—		**0.453**	**0.449**	*0.679*	*0.577*	*0.511*	*0.498*
ETTm1 → ETTh2	0.464	0.454	—		**0.396**	**0.413**	*0.452*	*0.441*	*0.417*	*0.422*
ETTm1 → ETTm2	0.300	0.337	—		**0.296**	**0.328**	*0.329*	*0.357*	*0.291*	*0.331*
ETTm2 → ETTm1	0.592	0.497	—		**0.533**	**0.471**	*0.554*	*0.478*	*0.427*	*0.448*
Anomaly detection and classification (averages)										
Anomaly F1 (%), 4 datasets	75.49		—		**79.96**		—		*85.75*	
Classification accuracy (%), 10 UEA datasets	**62.37**		—		60.90		—		*75.90*	

Table 3. Finished local runs per task, 11 October → 23 October 2026.

Task	Protocol	TimeMixer: then → now	TimeMixer++: then → now	SparseTSF: then → now
Long-term forecasting	8 datasets × 4 horizons = 32	30 → 30	21 → 21	20 → 32
M4	6 subsets	0 → 6	published only	6 → 6
PEMS	4 datasets	0 → 4	published only	4 → 4
Imputation	6 datasets × 4 mask rates = 24	20 → 24	published only	24 → 24
Few-shot	6 datasets × 4 horizons = 24	0 → 24	published only	23 → 24
Zero-shot	6 pairs × 4 horizons = 24	0 → 24	published only	24 → 24
Anomaly detection	5 datasets	0 → 4	published only	2 → 4
Classification	10 datasets	0 → 10	published only	10 → 10

Reading Table 2.

- **SparseTSF matches TimeMixer where its inductive bias fits.**
    - On long-term ETT (Avg) the two are level; SparseTSF wins both hourly ETT datasets (Appendix A).
    - On few-shot ETT it is clearly better.
    - It beats TimeMixer on all six zero-shot transfer pairs.

  It does this with 10^1–10^3 parameters against TimeMixer's 10^4–10^6. With so few parameters there is little to overfit when data are scarce or the target distribution shifts.

- **SparseTSF loses most where signal crosses channels.** On PEMS (traffic sensor networks) TimeMixer's MAE is half of SparseTSF's (17.51 against 34.13). On imputation TimeMixer is better on every row.

- **The local TimeMixer++ is undertrained** (Section 2.3): on ETTh1 its local MSE is 0.747 against 0.419 published (Appendix A). Its local column measures the reimplementation at a stable learning rate, not the architecture.

- **TimeMixer's anomaly average includes a failed run.** Its MSL F1 is 48.0 on seed 2021, against 80.2 and 79.4 on the two repeated seeds (Section 4.3). Single-seed anomaly averages should not be read as rankings.

2.3 Why the comparison cannot be completed on this setup

The tables still have gaps and protocol deviations. Each has a stated cause:

1. **TimeMixer++ is not run per task.** No official code exists. The reimplementation diverges (NaN losses) at the paper's learning rates, and the coordinated stability fixes (skipping non-finite steps, clamped attention and FFT amplitudes) only hold at `lr = 5e-5` with step decay. At that rate it is undertrained, so per-task runs would mainly measure the instability. Its per-task columns are therefore published values only.
2. **Memory and time on a 4 GB GPU.**
    - TimeMixer on Traffic at horizons 336 and 720 does not fit, which leaves Traffic partial.
    - TimeMixer++ was not run on Electricity and Traffic, and finished only horizon 96 on Solar-Energy.
    - Test-time metrics for Electricity at horizon 720 and Traffic at 336/720 had to be accumulated batch by batch (an opt-in mode verified against the default to 6–7 significant digits).
3. **SWaT is not obtainable.** It is gated behind the iTrust request form. A third-party copy exists but carries no redistribution right and lacks the standard attack-period test split, so anomaly detection covers 4 of 5 datasets. The published TimeMixer++ average in the tables is taken over the same four (85.76; 87.47 over all five).
4. **Imputation input length.** The TimeMixer++ paper uses an input of 1,024 steps. SparseTSF needs its period to divide the input length, and 1,024 admits only powers of two, which cannot align with the 24-

hour cycle of hourly data. All local imputation runs use 96 steps, and the published imputation numbers are not comparable.

5. **Configurations that do not exist.** TimeMixer publishes no anomaly-detection or classification settings; it runs there with its own long-term architecture, untuned. SparseTSF publishes no heads beyond forecasting; the heads used here were designed in this study (Section 1.4).
6. **Early stopping on test** in three upstream training loops (imputation, anomaly, classification). This is kept, because changing it would invalidate every existing number, and those results are labelled a sensitivity analysis.
7. **One seed for most cells.** Only the headline comparisons were repeated (Section 4.3); TimeMixer was repeated on PEMS and anomaly detection only, because its imputation alone takes about ten hours per seed on this machine.
8. **Energy** is measured with CodeCarbon. GPU power is measured (NVML); CPU and RAM power are estimates from rated power, since Windows exposes no RAPL counters. Ratios between models measured in the same session are reliable; absolute values and cross-session comparisons are not.

3. Modules tested on the SparseTSF baseline

Each module is a switchable component added to SparseTSF. Every module is tested under the same rules:

- **Off by default.** With every module off the model is bitwise identical to SparseTSF, which an automated check verifies after every change.
- **No dependence on the channel count.** No module adds a parameter whose size depends on the number of channels, so a trained model can be applied to a dataset with a different number of channels (the zero-shot task relies on this).
- **Validation decides.** Modules are chosen on validation loss.
- **Prediction first.** Each module's source, mechanism, prediction and refutation criterion were recorded in the progress log before its runs.

The modules are grouped by the part of SparseTSF they change. Table 4 summarises them; Section 4 describes how the adopted ones are combined into one model.

Table 4. Modules tested on SparseTSF.

#	Module	Part of the model	Borrowed from	Extra parameters	Status
3.1	Phase mixing	Forecasting core	TimeMixer++ dual-axis decomposition	p^2 (16–576)	**Adopted**
3.2	Trend–season decomposition	Forecasting core	Autoformer; TimeMixer	one period map	**Adopted** (learned trend)
3.3	Multi-period ensemble	Forecasting core	TimeMixer multi-scale ensemble	one map per period	Rejected
3.4	Period choice: phase-aligned folding, FFT selection	Forecasting core	TimeMixer++ padding; TimesNet FFT periods	0	Folding enables; FFT rule rejected
3.5	Instance normalisation (RevIN)	Forecasting core	Kim et al. 2022	0	Rejected (no effect)
3.6	Channel context (mean, correlation)	Forecasting core	SOFTS; iTransformer	2–9	Candidate (correlation)
3.7	Iterative refinement	Missing values	Iterative imputation (mice)	0	**Adopted**
3.8	Mask-aware aggregation	Missing values	Partial convolutions	0	**Adopted**
3.9	Phase mixing on incomplete inputs	Missing values	Partial convolutions	0	Candidate (mask-aware); complete-only rejected
3.10	Leave-one-out reconstruction	Missing values / anomaly	Noise2Void; Noise2Self	0	Rejected

#	Module	Part of the model	Borrowed from	Extra parameters	Status
3.11	Learned gates	Safe defaults for other modules	ReZero	1 per gated module	Refinement gate **adopted**; phase gate rejected
3.12	Pooled classification readouts	Classification head	x-vectors; piecewise aggregate approximation	smaller than flatten	Rejected

Forecasting core

The forecasting core is SparseTSF's own mechanism: fold the input by a period, aggregate with a small convolution, and map periods to future periods with one linear map per phase.

3.1 Phase mixing

- **Why.** SparseTSF's forecast map, written per channel, is a Kronecker product `I_p` ⊗ `W_n`: one small map across periods, applied independently at each of the `p` phases. The shape of the signal *within* a period is never modelled.
- **Where from.** TimeMixer++'s dual-axis "time image" decomposition, which attends along both the period axis and the phase axis. It is reduced here to one shared linear map along the phase axis.
- **What.** A second linear map across phases turns the forecast into `W_p` ⊗ `W_n`, a factorised full map. It costs `p²` parameters (16 at p = 4, 576 at p = 24) independent of input length, against `seq_len × pred_len` for a dense map. It is zero-initialised, so training starts from SparseTSF.
- **Result.**
  - **PEMS:** at period 4 it cut the MAE on all four datasets from 34.13 to 25.50 (−25%) with 93 parameters, within 0.6% of a dense map with 12× the parameters.
  - **ETTh1:** it overfits (+2.7%, worse on validation as well). A rank-4 factorisation removes most of the overfit (0.394 → 0.387) but never beats the plain module.
  - **Imputation:** over three seeds its effect is within noise (Section 4.3).
- **Lesson.** Within-period shape is the main thing SparseTSF misses on traffic data, and a factorised map recovers nearly all of a dense model's accuracy at a small fraction of its size. Its weakness is small datasets. **Adopted.**

3.2 Trend–season decomposition

- **Why.** Drift over the input window is something a period-wise map extrapolates poorly.
- **Where from.** The moving-average decomposition of Autoformer (Wu et al., NeurIPS 2021), also used by TimeMixer.
- **What.** Split the input into a moving-average trend and a seasonal remainder; the seasonal part goes through the period map. Two trend treatments were tested:
  - holding the trend at its last value;
  - giving the trend its own period map.
- **Result.**
  - **Flat trend:** harmful on every dataset (+13% to +27% MSE). It removes drift that SparseTSF could already model.
  - **Learned trend:** the best forecasting module on validation on all three datasets screened. It improves ETTm1 at all horizons (−0.8% to −2.7%) and Weather (−0.2% to −1.0%), is mixed on ETTh1, and doubles the period map (581 → 1,157 parameters on ETTm1).
- **Lesson.** A decomposition helps only if the trend gets its own predictor. **Adopted** (learned trend).

3.3 Multi-period ensemble

- **Why.** One hand-chosen period is a strong assumption about which cycle dominates.
- **Where from.** TimeMixer's ensemble over temporal scales, applied to periods instead of resolutions.
- **What.** Run the fold at several periods and add the branches behind zero-initialised gates.
- **Result.** Small and inconsistent: −1.8% on ETTm1, +0.7% on ETTh1, +0.5% on Weather. It never had the lowest validation loss.
- **Lesson.** One well-chosen period already captures what extra periods add here. **Rejected.**

3.4 Period choice: phase-aligned folding and FFT-based selection

- **Why.** The period decides how many segments SparseTSF sees, which is all of its capacity, yet it is a fixed value from the SparseTSF paper (24 for hourly data, 4 for 15- and 10-minute data, 12 for PEMS). A sweep of periods, chosen on validation, showed how much it matters:

  - **ETTh1:** prefers period 12 to the published 24 (−3.2% MSE at horizon 96), which turns a 2.8% loss against TimeMixer into a 0.5% win;
  - **PEMS08:** prefers period 1, no folding at all (−24% MAE);
  - **Weather and ETTm1:** insensitive.

  Choosing per dataset is tuning, so two modules aim at a general rule.

- **Where from.**

  - TimeMixer++'s pad-and-fold, corrected so that every forecast step keeps its phase.
  - TimesNet (Wu et al., ICLR 2023) and TimeMixer++, which take periods from the strongest FFT frequencies.

- **What.**

  - **Phase-aligned folding** pads the front of the window, so periods that do not divide the input length become usable without shifting the phase of any forecast step.
  - **FFT selection** takes, on each training split, the strongest period between 2 and 48 steps, snapped to a divisor of 96.

- **Result.** Folding works as intended and removes the divisibility constraint. The FFT rule picks the published 24 on ETTh1 and ETTh2, and 48 on every other dataset. The 48 is the edge of the allowed band, because the spectra are dominated by slow variation, and the sweep had already measured it as worse (+6.9% on ETTm1, +7.7% on Weather). No training was needed.

- **Lesson.** The strongest cycle in the data is the wrong target for SparseTSF, which benefits from short periods with more segments per window. Folding is kept as an enabler; the FFT rule is **rejected**. The fixed configuration keeps the published periods.

3.5 Instance normalisation (RevIN)

- **Why.** RevIN (Kim et al., ICLR 2022) is a standard remedy for distribution shift in forecasting.
- **What.** Divide each window by its standard deviation and multiply the forecast back.
- **Result.** It provably has no effect on SparseTSF. Its forecast map is linear and bias-free, hence $f(x/\sigma)\cdot\sigma = f(x)$; measured, the outputs differ by 1.5×10^{-5} (floating-point rounding).
- **Lesson.** Any per-window rescaling is inert on SparseTSF, so a whole family of modules can be ruled out without training. **Rejected.**

3.6 Channel context

- **Why.** PEMS is the largest remaining gap to TimeMixer. Neighbouring traffic sensors move together, SparseTSF treats every channel independently, and TimeMixer runs PEMS with its channels embedded jointly.
- **Where from.**
  - The aggregate-and-redistribute core of SOFTS (Han et al., NeurIPS 2024).
  - Attention across variates in iTransformer (Liu et al., ICLR 2024), with its learned projections removed.
- **What.** Both forms add a gated cross-channel term to each forecast, without any parameter that depends on the channel count:
  - **Channel mean:** the mean of all channels gets its own small map, added to each channel in proportion to that channel's loading on the mean.
  - **Channel correlation:** each channel adds a softmax-weighted mix of the other channels' forecasts, weighted by input correlation, with a learned temperature (2 parameters).
- **Result.**
  - **One seed:** PEMS MAE −3.9% (mean) and −4.9% (correlation), with no harm on long-term guard datasets. Neither reached the pre-registered 5% validation threshold for a full-suite run.
  - **Three seeds:** both gains are findings (mean −3.1% ± 0.7%; correlation −5.25% ± 0.29%).
  - PEMS remains about 48% behind TimeMixer.
- **Lesson.** Cross-channel information helps a little at almost no cost, but TimeMixer's PEMS lead needs deeper channel interaction. A screening threshold applied to one seed rejected a small effect that proved real. Channel correlation is a **candidate** awaiting a full-suite run.

Missing values and reconstruction

These modules change how SparseTSF handles inputs with missing values (imputation) and how it reconstructs its input (anomaly detection).

3.7 Iterative refinement

- **Why.** Imputation is scored only on missing positions. At a mask rate of 0.5, half of the convolution's window is literal zeros that the model cannot distinguish from observations, and SparseTSF's error grew monotonically with the mask rate.
- **Where from.** The loss definition itself, and the tradition of iterative imputation (for example chained equations, `mice`). The refinement step is the cheapest form of it.
- **What.** Run the model once, fill the holes with its estimate, recompute the normalisation on the now complete series, and run the *same* weights again. No new parameters; one extra forward pass.
- **Result.** The largest single gain in the study. On ETTh1 imputation it cut the error by 26% (0.1633 → 0.1207) with no added parameter; the six-dataset mean fell from 0.0838 to 0.0733. On ETTh2 it made results worse (0.0703 → 0.0955), which the gate of Section 3.11 later repaired.
- **Lesson.** The most effective module costs nothing; it changes how the existing model is used. **Adopted**, with its gate.

3.8 Mask-aware aggregation

- **Why.** Same diagnosis as Section 3.7: missing values enter the aggregating convolution as zeros.
- **Where from.** Partial convolutions for image inpainting (Liu et al., ECCV 2018).
- **What.** Compute the convolution from observed values only, and rescale by the share of the window that was observed.
- **Result.**
  - **Alone:** no effect (ETTh1, mask 0.5: 0.2141 → 0.2136).

    - **With refinement:** it helps on large datasets (ETTm1: 0.0497 → 0.0426) and hurts slightly on small ones (ETTh1: 0.1207 → 0.1263).
- **Lesson.** The two missing-value modules are complementary, and how much capacity pays off depends on training-set size (about 34,000 windows for ETTm1 against 8,500 for ETTh1). **Adopted.**

3.9 Phase mixing on incomplete inputs

- **Why.** With refinement gated, phase mixing appeared to cost accuracy in imputation, only at high mask rates (0.375 and 0.5). That pattern suggested phase mixing spreads the holes along the phase axis.
- **Where from.** Partial convolutions again (Section 3.8).
- **What.** Two forms:
    - **Complete inputs only:** skip phase mixing in the first pass, which still has the holes.
    - **Mask-aware:** mix only observed positions and rescale.
- **Result.**
    - **Complete inputs only:** rejected (+23–42% on Electricity, +29–39% on ETTh1, even at low mask rates).
    - **Mask-aware:** on one seed it brought the imputation error within 1.0% of the model without phase mixing. Over three seeds the difference to plain phase mixing is within noise (−2.3% ± 2.1% on ETTh1/ETTh2, −6.0% ± 7.4% on Electricity). Its one consistent effect: on Electricity at mask 0.5 it is the best variant on all three seeds.
- **Lesson.** The first pass needs phase mixing. The cost this module targeted was itself largely run-to-run noise (Section 4.3). Mask-aware mixing is a **candidate**, unconfirmed.

3.10 Leave-one-out reconstruction

- **Why.** On SMD, SparseTSF's anomaly detector was limited by recall (73.9) rather than precision (87.3). A reconstruction model can represent the identity map exactly, and copying the input reconstructs anomalies faithfully too.
- **Where from.** Blind-spot denoising (Noise2Void, Krull et al., CVPR 2019; Noise2Self, Batson and Royer, ICML 2019): predict each point without seeing it.
- **What.** Remove the self-term of the reconstruction map.
- **Result.** SMD recall improved slightly (73.90 → 75.39; F1 80.02 → 80.55), but PSM got worse (F1 95.26 → 93.23).
- **Lesson.** Copying the input explains little of SMD's gap. **Rejected.**

Safe defaults

3.11 Learned gates

- **Why.** A module that helps one dataset and hurts another cannot be on everywhere unless it has a safe default. Refinement hurt ETTh2, and phase mixing appeared to hurt Electricity imputation. Both raised the *training* loss as well, so the harm was visible to training rather than being overfitting.
- **Where from.** ReZero (Bachlechner et al., UAI 2021): a residual branch multiplied by one scalar initialised at zero. The multi-period ensemble already used such gates.
- **What.** Each gate is one learned number per module, starting at zero, so the model starts exactly as if the module were off (verified bitwise).
    - **Refinement:** the output becomes pass 1 + g·(pass 2 − pass 1).
    - **Phase mixing:** the phase image becomes x + g·W_p·x.
- **Result.**

    - **Refinement gate:** the six-dataset imputation mean fell from 0.0716 to 0.0651, better on all six datasets. On ETTh2 the gate settled at about 0.5 (averaging the two passes) and repaired the failure (0.0955 → 0.0641). It settled near 1 on the larger datasets.
    - **Phase gate:** no effect (0.0688 against 0.0689).
- **Lesson.** Letting a module be partly on can repair it, and the learned values are interpretable. A gate cannot block harm that training actively moves into. The refinement gate is **adopted**; the phase gate is **rejected**. The ungated comparison was not repeated over seeds.

Classification head

3.12 Pooled classification readouts

- **Why.** The flatten readout scales with input length (up to 970,712 parameters) and has no invariance to time shifts.
- **Where from.**
    - Statistics pooling in speaker recognition (x-vectors, Snyder et al., ICASSP 2018).
    - Segment means as in piecewise aggregate approximation (Keogh et al., 2001).
- **What.** Four variants:
    - per-channel mean, standard deviation and maximum over time;
    - segment means, which keep coarse time order;
    - a wider aggregator;
    - a fix for a padding bias in the centring step.
- **Result.** The ten-dataset average fell from 60.90 to 36.69 with global statistics pooling. Shape-driven datasets collapsed (UWaveGestureLibrary 72.8 → 9.4), and segment pooling did not recover them. One dataset gained strongly: Heartbeat 54.6 → 71.7 with a readout 134× smaller, where the flatten readout had a 242:1 parameter-to-sample ratio.
- **Lesson.** Pooling helps where classes differ in amplitude statistics and fails where they differ in shape. The classification gap to TimeMixer++ (15 points, published) lies in what the backbone extracts, not in the readout. **Rejected.**

4. SparseTSFPlus: the adopted modules as one model

4.1 One fixed configuration

The thesis target is one model: one configuration run unchanged on every task and dataset, as TimeMixer++ is one architecture with thin task heads. **SparseTSFPlus** is SparseTSF with the adopted modules of Section 3, fixed for everything:

- **Forecasting core:** phase mixing (3.1) and the learned-trend decomposition (3.2), with SparseTSF's published period on every dataset.
- **Inputs with missing values:** iterative refinement behind its gate (3.7, 3.11) and mask-aware aggregation (3.8).
- **Heads:** the task heads of Section 1.4.

To measure what this generality costs, a **per-task** version is also reported. It uses, for each task, the set of modules that won that task's ablation, applied unchanged to all of its datasets. It is a reference point, not the model.

4.2 Comparison

Table 5. SparseTSFPlus (fixed configuration) against the baselines (seed 2021, same layout as Tables 1–2). The per-task version and every dataset are in Appendix A.

Benchmark	TimeMixer		SparseTSF		SparseTSFPlus		*TimeMixer (paper)*		*TimeMixer++ (paper)*	
	MSE	**MAE**	**MSE**	**MAE**	**MSE**	**MAE**	**MSE**	**MAE**	**MSE**	**MAE**
Long-term forecasting (MSE / MAE, mean of 4 horizons, input 96)										
ETT (Avg)	0.374	0.393	0.374	0.389	**0.370**	**0.386**	*0.367*	*0.389*	*0.349*	*0.377*
Weather	**0.245**	**0.276**	0.274	0.294	0.273	0.293	*0.240*	*0.272*	*0.226*	*0.262*
Solar-Energy	**0.232**	**0.298**	0.386	0.371	0.365	0.358	*0.216*	*0.280*	*0.203*	*0.258*
Electricity	**0.183**	**0.272**	0.222	0.296	0.219	0.296	*0.182*	*0.273*	*0.165*	*0.253*
Traffic	partial		0.635	**0.379**	**0.633**	0.380	*0.485*	*0.297*	*0.416*	*0.264*
Univariate short-term forecasting, M4 (average of 6 subsets)										
SMAPE	**11.718**		13.436		12.798		*11.723*		*11.448*	
MASE	**1.563**		1.825		1.690		*1.559*		*1.487*	
OWA	**0.841**		0.972		0.914		*0.840*		*0.821*	
Multivariate short-term forecasting, PEMS (average of 4 datasets, 96 → 12)										
MAE	**17.51**		34.13		25.45		*17.41*		*15.91*	
MAPE (%)	**11.41**		20.81		15.10		*10.59*		*10.08*	
RMSE	**28.14**		52.10		40.55		*28.01*		*27.06*	
Imputation (MSE / MAE, mean of 4 mask rates, input 96)										
ETT (Avg)	**0.065**	**0.155**	0.084	0.175	0.067	0.155	n/c		n/c	
Electricity	**0.080**	**0.192**	0.130	0.259	0.116	0.233	n/c		n/c	
Weather	0.032	0.050	0.034	0.058	**0.030**	**0.045**	n/c		n/c	
Few-shot forecasting, 10% of training data (MSE / MAE, mean of 4 horizons)										
ETT (Avg)	0.513	0.462	0.441	0.427	**0.429**	**0.419**	*0.453*	*0.445*	*0.396*	*0.421*
Weather	**0.254**	**0.278**	0.271	0.298	0.269	0.294	*0.242*	*0.281*	*0.241*	*0.271*
Electricity	**0.209**	**0.292**	0.224	0.297	0.226	0.304	*0.187*	*0.277*	*0.168*	*0.271*
Zero-shot transfer, train → test (MSE / MAE, mean of 4 horizons)										
ETTh1 → ETTh2	0.367	0.393	0.360	0.390	**0.355**	**0.387**	*0.427*	*0.424*	*0.367*	*0.391*
ETTh1 → ETTm2	0.327	0.367	0.321	0.359	**0.318**	**0.358**	*0.361*	*0.397*	*0.301*	*0.357*
ETTh2 → ETTh1	0.639	0.548	**0.453**	**0.449**	0.463	0.455	*0.679*	*0.577*	*0.511*	*0.498*
ETTm1 → ETTh2	0.464	0.454	0.396	0.413	**0.394**	**0.412**	*0.452*	*0.441*	*0.417*	*0.422*
ETTm1 → ETTm2	0.300	0.337	0.296	0.328	**0.294**	**0.326**	*0.329*	*0.357*	*0.291*	*0.331*
ETTm2 → ETTm1	0.592	0.497	0.533	0.471	**0.522**	**0.466**	*0.554*	*0.478*	*0.427*	*0.448*
Anomaly detection and classification (averages)										
Anomaly F1 (%), 4 datasets	75.49		79.96		**79.98**		—		*85.75*	
Classification accuracy (%), 10 UEA datasets	**62.37**		60.90		60.90		—		*75.90*	

4.3 Over three seeds

Table 6 gives the mean ± standard deviation over seeds 2021–2023 of each task's headline number, for every configuration that was repeated. The comparisons that are findings under the rule of Section 1.3:

- **Fixed vs SparseTSF:** PEMS −24.4% ± 0.9%, imputation (ETTh1/ETTh2) −16.2% ± 2.6%, M4 −6.1% ± 0.6%, few-shot −1.4% ± 0.5%, long-term −1.0% ± 0.2%.
- **Fixed vs per task:** long-term −0.4% ± 0.1% (fixed better); zero-shot +0.2% ± 0.1% (per task better).
- **TimeMixer vs fixed:** PEMS, by 47.7% ± 2.0%.

Not findings:

- **Fixed vs SparseTSF:** zero-shot (−0.42% ± 0.24%) and anomaly detection (+0.02 F1).
- **Fixed vs TimeMixer on anomaly detection:** +4.5, −2.5 and −2.9 F1 by seed.
- **Every imputation difference between SparseTSFPlus variants.**

The repeats corrected three single-seed claims:

- **Phase mixing's cost in imputation.** It was +20.1% on ETTh2 on seed 2021, but −2.7% and −2.0% on the other seeds; the bad ETTh2 runs are a training plateau that also strikes the model without phase mixing.
- **The mask-aware fix (3.9)** is within noise.
- **The anomaly-detection win over TimeMixer** came from TimeMixer's failed seed-2021 MSL run.

Forecasting results are very stable (0.1–0.5% spread between seeds); imputation varies by 4–7%.

Table 6. Mean ± standard deviation over seeds 2021, 2022 and 2023. The per-task imputation form is gated refinement; "—" means not repeated.

Task	SparseTSF	SparseTSFPlus fixed	SparseTSFPlus per task	TimeMixer
Long-term (ETT×4, Weather; 4 horizons) (MSE ↓)	0.3538 ± 0.0007	0.3503 ± 0.0003	0.3516 ± 0.0002	—
M4 (6 subsets) (OWA ↓)	0.973 ± 0.004	0.914 ± 0.002	0.921 ± 0.005	—
PEMS (4 datasets) (MAE ↓)	34.12 ± 0.01	25.79 ± 0.30	—	17.46 ± 0.06
Few-shot (6 datasets) (MSE ↓)	0.3747 ± 0.0014	0.3694 ± 0.0007	0.3707 ± 0.0020	—
Zero-shot (6 pairs) (MSE ↓)	0.3931 ± 0.0011	0.3914 ± 0.0003	0.3905 ± 0.0003	—
Anomaly (4 datasets) (F1 ↑)	80.47 ± 0.85	80.49 ± 0.53	—	80.81 ± 4.61
Imputation (ETTh1, ETTh2; 4 mask rates) (MSE ↓)	0.1151 ± 0.0015	0.0965 ± 0.0035	0.0947 ± 0.0012	—
Imputation (Electricity; 4 mask rates) (MSE ↓)	—	0.1067 ± 0.0079	0.1007 ± 0.0044	—

4.4 Model size

Table 7. Trainable parameters (range over datasets and horizons).

Task	SparseTSF	SparseTSFPlus (fixed)	TimeMixer	TimeMixer++
Long-term forecasting (seq_len 96)	41–4,325	633–8,661	75,497–2,511,077	112,951–233,885
Short-term forecasting, M4	13–33	37–617	10,525–100,069	—
Short-term forecasting, PEMS	21	173	869,170–1,237,791	—
Imputation	41–581	598–618	57,833–60,345	—
Anomaly detection	41	666	62,573–62,813	—
Classification	3,142–970,712	3,142–970,712	14,349–18,227,514	—

4.5 Energy

The matched energy protocol trains every model with the same settings: 5 epochs, no early stopping, batch 64. It runs long-term forecasting on five datasets, reporting the total over four horizons in kWh. Its accuracy is not comparable with the tables above. Short runs vary between sessions (the SparseTSF total measured in July was 42% higher than in October), so SparseTSF, SparseTSFPlus and TimeMixer were measured in one session. TimeMixer++ comes from an earlier session and is indicative only.

Table 8. Energy per dataset (kWh, total over four horizons).

Dataset	SparseTSF	SparseTSFPlus (fixed)	SparseTSFPlus (per task)	TimeMixer	TimeMixer++
ETTh1	0.00037	0.00040	0.00041	0.00425	0.01176

Dataset	SparseTSF	SparseTSFPlus (fixed)	SparseTSFPlus (per task)	TimeMixer	TimeMixer++
ETTh2	0.00038	0.00041	0.00041	0.00440	0.01024
ETTm1	0.00104	0.00138	0.00136	0.01865	0.04026
ETTm2	0.00109	0.00139	0.00138	0.01879	0.03825
Weather	0.00162	0.00248	0.00242	0.03903	0.04891
Total	**0.00450**	**0.00606**	**0.00598**	**0.08511**	**0.14941**
× SparseTSF	*1.00*	*1.35*	*1.33*	*18.91*	*33.19*
Measured	*2026-10-21*	*2026-10-21*	*2026-10-21*	*2026-10-21*	*2026-07-23*

4.6 Discussion

SparseTSFPlus answers half of the thesis question. It is efficient: 10^1–10^3 parameters outside classification, and 1.35× SparseTSF's energy, about 7% of TimeMixer's. With one configuration it improves on SparseTSF beyond run-to-run noise on five of the eight tasks, and choosing modules per task adds nothing measurable on forecasting. It leads TimeMixer on few-shot and zero-shot forecasting, where small models generalise better. It is not yet as capable as TimeMixer on three fronts, each with a known cause:

- **PEMS and other cross-channel data:** the model is channel-independent by design. Channel context recovers about 5%, against a gap of about 48%.
- **Long input dependence:** Weather and Solar have daily cycles longer than the 96-step input, which no period-based module can recover.
- **Classification:** neither SparseTSF's backbone nor any readout tested reaches TimeMixer++'s published accuracy.

5. Conclusions and next steps

1. **One efficient model is feasible.** A fixed SparseTSFPlus configuration of at most 8,661 parameters outside classification improves on SparseTSF on five tasks beyond seed noise, at no measurable cost against per-task choice.
2. **The cheapest modules did the most:** iterative refinement, its one-parameter gate, and phase mixing. Several ideas from larger models did not transfer: RevIN, FFT period selection, the multi-period ensemble, leave-one-out reconstruction, and pooled readouts.
3. **Seed repeats are necessary on imputation and anomaly detection.** Three single-seed claims did not survive them.

Decisions for discussion.

- Run channel correlation on the full suite. It is confirmed on PEMS over seeds, but was held back by the one-seed screen.
- Make imputation comparisons paired (the same masks for every variant).
- Decide how to address cross-channel structure inside the backbone without breaking transfer between datasets.

References

Cited from memory; titles, venues and years should be checked before citing in the thesis.

- Bachlechner, T. et al. ReZero is All You Need: Fast Convergence at Large Depth. UAI 2021.
- Batson, J. and Royer, L. Noise2Self: Blind Denoising by Self-Supervision. ICML 2019.

- Han, L. et al. SOFTS: Efficient Multivariate Time Series Forecasting with Series-Core Fusion. NeurIPS 2024.
- Keogh, E. et al. Dimensionality Reduction for Fast Similarity Search in Large Time Series Databases. Knowledge and Information Systems, 2001.
- Kim, T. et al. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. ICLR 2022.
- Krull, A., Buchholz, T.-O. and Jug, F. Noise2Void — Learning Denoising from Single Noisy Images. CVPR 2019.
- Lin, S. et al. SparseTSF: Modeling Long-term Time Series Forecasting with 1k Parameters. ICML 2024.
- Liu, G. et al. Image Inpainting for Irregular Holes Using Partial Convolutions. ECCV 2018.
- Liu, Y. et al. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. ICLR 2024.
- Snyder, D. et al. X-Vectors: Robust DNN Embeddings for Speaker Recognition. ICASSP 2018.
- van Buuren, S. and Groothuis-Oudshoorn, K. mice: Multivariate Imputation by Chained Equations in R. Journal of Statistical Software, 2011.
- Wang, S. et al. TimeMixer: Decomposable Multiscale Mixing for Time Series Forecasting. ICLR 2024.
- Wang, S. et al. TimeMixer++: A General Time Series Pattern Machine for Universal Predictive Analysis. ICLR 2025.
- Wu, H. et al. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. NeurIPS 2021.
- Wu, H. et al. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. ICLR 2023.

Appendix A. Full results per dataset (23 October 2026)

Every dataset of every task, for all models, at seed 2021. Each value is the mean over the four horizons (forecasting) or four mask rates (imputation), or the task's own metric. The best local value per row is in **bold**; published values are in *italics*. "partial (k/4)" marks a dataset where only k of its four cells finished.

Table A1. All models, every dataset.

Dataset	TimeMixer	TimeMixer++	SparseTSF	SparseTSFPlus fixed	SparseTSFPlus per task	TimeMixer (paper)	TimeMixer++ (paper)
Long-term (MSE ↓)							
ETTh1	0.4578	0.7467	0.4347	**0.4291**	0.4318	*0.4465*	*0.4185*
ETTh2	0.3730	0.4507	0.3566	**0.3509**	0.3551	*0.3648*	*0.3390*
ETTm1	**0.3841**	0.4579	0.4172	0.4132	0.4109	*0.3812*	*0.3685*
ETTm2	**0.2799**	0.3125	0.2884	0.2868	0.2866	*0.2752*	*0.2687*
Weather	**0.2454**	0.2756	0.2741	0.2727	0.2726	*0.2403*	*0.2263*
Solar-Energy	**0.2319**	partial (1/4)	0.3858	0.3649	0.3643	*0.2162*	*0.2032*
Electricity	**0.1830**	—	0.2218	0.2189	0.2198	*0.1822*	*0.1645*
Traffic	partial (2/4)	—	0.6346	**0.6329**	0.6349	*0.4848*	*0.4158*
M4 (OWA ↓)							
M4, 6 subsets (OWA)	**0.841**	—	0.972	0.914	0.918	*0.840*	*0.821*
PEMS (MAE ↓)							
PEMS03	**14.81**	—	28.28	21.25	21.34	—	—
PEMS04	**19.22**	—	36.12	27.37	27.43	—	—
PEMS07	**20.69**	—	42.89	30.94	31.02	—	—
PEMS08	**15.33**	—	29.25	22.24	22.22	—	—
Average	**17.51**	—	34.13	25.45	25.50	*17.41*	*15.91*
Imputation (MSE ↓)							

Dataset	TimeMixer	TimeMixer++	SparseTSF	SparseTSFPlus fixed	SparseTSFPlus per task	TimeMixer (paper)	TimeMixer++ (paper)
ETTh1	**0.1201**	—	0.1633	0.1222	0.1230	—	—
ETTh2	0.0664	—	0.0703	0.0770	**0.0641**	—	—
ETTm1	0.0470	—	0.0715	**0.0415**	0.0420	—	—
ETTm2	0.0278	—	0.0327	**0.0268**	0.0269	—	—
Weather	0.0315	—	0.0344	0.0300	**0.0299**	—	—
Electricity	**0.0803**	—	0.1304	0.1158	0.1048	—	—
Few-shot (MSE ↓)							
ETTh1	0.6865	—	0.5517	**0.5251**	0.5337	—	—
ETTh2	0.4550	—	0.4297	0.4173	**0.4169**	—	—
ETTm1	0.6098	—	0.4935	0.4884	**0.4850**	—	—
ETTm2	0.3018	—	0.2885	0.2867	**0.2867**	—	—
ETT average	0.5133	—	0.4408	**0.4294**	0.4306	*0.4530*	*0.3960*
Weather	**0.2536**	—	0.2708	0.2691	0.2686	*0.2420*	*0.2410*
Electricity	**0.2093**	—	0.2238	0.2259	0.2217	*0.1870*	*0.1680*
Zero-shot (MSE ↓)							
ETTh1 → ETTh2	0.3669	—	0.3596	**0.3554**	0.3561	*0.4270*	*0.3670*
ETTh1 → ETTm2	0.3268	—	0.3207	**0.3184**	0.3196	*0.3610*	*0.3010*
ETTh2 → ETTh1	0.6387	—	**0.4527**	0.4629	0.4544	*0.6790*	*0.5110*
ETTm1 → ETTh2	0.4640	—	0.3958	0.3944	**0.3939**	*0.4520*	*0.4170*
ETTm1 → ETTm2	0.2998	—	0.2955	**0.2938**	0.2939	*0.3290*	*0.2910*
ETTm2 → ETTm1	0.5917	—	0.5329	**0.5223**	0.5254	*0.5540*	*0.4270*
Anomaly (F1 % ↑)							
PSM	**96.72**	—	95.26	95.38	95.26	—	*97.60*
SMD	**84.18**	—	80.02	80.03	80.02	—	*86.50*
MSL	48.02	—	**77.80**	77.79	77.80	—	*85.82*
SMAP	**73.05**	—	66.76	66.74	66.76	—	*73.10*
SWaT (not obtainable)	—	—	—	—	—	—	*94.64*
Classification (accuracy % ↑)							
EthanolConcentration	27.00	—	**29.28**	29.28	29.28	—	—
FaceDetection	**70.03**	—	68.59	68.59	68.59	—	—
Handwriting	6.35	—	**8.59**	8.59	8.59	—	—
Heartbeat	**75.61**	—	54.63	54.63	54.63	—	—
JapaneseVowels	46.49	—	**61.62**	61.62	61.62	—	—
PEMS-SF	78.61	—	**84.39**	84.39	84.39	—	—
SelfRegulationSCP1	**90.78**	—	82.25	82.25	82.25	—	—
SelfRegulationSCP2	**56.11**	—	52.78	52.78	52.78	—	—
SpokenArabicDigits	**95.54**	—	94.04	94.04	94.04	—	—
UWaveGestureLibrary	**77.19**	—	72.81	72.81	72.81	—	—
Average	**62.37**	—	60.90	60.90	60.90	—	*75.90*

Appendix B. Full results per dataset on 11 October 2026

The same per-dataset view of the state summarised in Table 1, read from git at commit `72040e6`.

Table B1. Three models, every dataset, as of 11 October 2026.

Dataset	TimeMixer	TimeMixer++	SparseTSF	TimeMixer (paper)	TimeMixer++ (paper)
Long-term (MSE ↓)					
ETTh1	0.4578	0.7467	**0.4347**	*0.4465*	*0.4185*
ETTh2	0.3730	0.4507	**0.3566**	*0.3648*	*0.3390*
ETTm1	**0.3841**	0.4579	0.4172	*0.3812*	*0.3685*
ETTm2	**0.2799**	0.3125	0.2884	*0.2752*	*0.2687*
Weather	**0.2454**	0.2756	0.2741	*0.2403*	*0.2263*
Solar-Energy	0.2319	partial (1/4)	—	*0.2162*	*0.2032*
Electricity	0.1830	—	—	*0.1822*	*0.1645*
Traffic	partial (2/4)	—	—	*0.4848*	*0.4158*
M4 (OWA ↓)					
M4, 6 subsets (OWA)	—	—	0.972	*0.840*	*0.821*
PEMS (MAE ↓)					
PEMS03	—	—	28.28	—	—
PEMS04	—	—	36.12	—	—
PEMS07	—	—	42.89	—	—
PEMS08	—	—	29.25	—	—
Average	—	—	34.13	*17.41*	*15.91*
Imputation (MSE ↓)					
ETTh1	**0.1201**	—	0.1633	—	—
ETTh2	**0.0664**	—	0.0703	—	—
ETTm1	**0.0470**	—	0.0715	—	—
ETTm2	**0.0278**	—	0.0327	—	—
Weather	**0.0315**	—	0.0344	—	—
Electricity	—	—	0.1304	—	—
Few-shot (MSE ↓)					
ETTh1	—	—	0.5517	—	—
ETTh2	—	—	0.4297	—	—
ETTm1	—	—	0.4935	—	—
ETTm2	—	—	0.2885	—	—
ETT average	—	—	0.4408	*0.4530*	*0.3960*
Weather	—	—	0.2708	*0.2420*	*0.2410*
Electricity	—	—	partial (3/4)	*0.1870*	*0.1680*
Zero-shot (MSE ↓)					
ETTh1 → ETTh2	—	—	0.3596	*0.4270*	*0.3670*
ETTh1 → ETTm2	—	—	0.3207	*0.3610*	*0.3010*
ETTh2 → ETTh1	—	—	0.4527	*0.6790*	*0.5110*
ETTm1 → ETTh2	—	—	0.3958	*0.4520*	*0.4170*
ETTm1 → ETTm2	—	—	0.2955	*0.3290*	*0.2910*
ETTm2 → ETTm1	—	—	0.5329	*0.5540*	*0.4270*
Anomaly (F1 % ↑)					

Dataset	TimeMixer	TimeMixer++	SparseTSF	TimeMixer (paper)	TimeMixer++ (paper)
PSM	—	—	95.26	—	*97.60*
SMD	—	—	80.02	—	*86.50*
MSL	—	—	—	—	*85.82*
SMAP	—	—	—	—	*73.10*
SWaT (not obtainable)	—	—	—	—	*94.64*
Classification (accuracy % ↑)					
EthanolConcentration	—	—	29.28	—	—
FaceDetection	—	—	68.59	—	—
Handwriting	—	—	8.59	—	—
Heartbeat	—	—	54.63	—	—
JapaneseVowels	—	—	61.62	—	—
PEMS-SF	—	—	84.39	—	—
SelfRegulationSCP1	—	—	82.25	—	—
SelfRegulationSCP2	—	—	52.78	—	—
SpokenArabicDigits	—	—	94.04	—	—
UWaveGestureLibrary	—	—	72.81	—	—
Average	—	—	60.90	—	*75.90*

Appendix C. Reproducing this report

All runners are idempotent: a configuration whose log already holds its result is skipped. The tables are regenerated, and the PDF rebuilt (pandoc and Chrome or Edge), by:

```
cd TimeMixer
venv/Scripts/python.exe benchmarks/tools/build_progress_document.py --pdf
```

Detailed per-task reports are in `TimeMixer/reports/`:

- `results_comparison_<task>.md` for each task's comparison;
- `results_sparsetsf_plus_report.md` for the modules;
- `results_sparsetsf_plus_suite.md` for the one-model runs;
- `results_seed_repeats.md` for the seed repeats.

The dated record of every step, with its plan and prediction written before its runs, is `reports/progress_log.md`.